

Provenance-based APT campaigns detection via masked graph representation learning

Jiafeng Ren, Rong Geng*

School of Computer Science and Technology, Northeastern University, Shenyang, China

ARTICLE INFO

Keywords:

Advanced Persistent Threat
Self-supervised learning
Provenance data

ABSTRACT

Advanced Persistent Threats (APTs) are well-planned, persistent, and highly stealthy cyberattacks designed to steal confidential information or disrupt specific target systems. Recent studies have used system audit logs to construct provenance graphs that describe system interactions to detect potentially malicious activities. Although they are effective, they still suffer from problems such as the need for a priori knowledge, lack of attack data, and high computational overhead that limit their application. In this paper, we propose a self-supervised learning-based APT detection model, APT-MGL, which learns the embedded representations of nodes through a graph mask self-encoder and transforms the detection problem into an outlier detection problem for malicious nodes. APT-MGL characterizes the behavior of nodes based on node type, action, and interaction frequency, and fuses the features through a multi-head self-attention mechanism. Then the node embedding is obtained by combining graph features and structural information using masked graph representation learning. Finally, the unsupervised outlier detection method is used to analyze the computed embeddings and obtain the final detection results. The experimental results show that APT-MGL outperforms existing monitoring models and achieves a small overhead.

1. Introduction

Advanced Persistent Threats (APTs) are sophisticated and persistent cyberattacks, usually orchestrated by certain individuals against specific targets. Such threats are often motivated by commercial or political purposes, designed to target specific organizations or countries, and require a high degree of stealth over an extended period of time (Alshamrani et al., 2019). APT attackers are usually an organization whose attacks tend to be well-organized, targeted, highly sophisticated, and stealthy. Attackers spend a lot of time and resources researching and identifying weaknesses within the target system, and the penetration process and data exfiltration phase can last for months or even years. Such attacks often target organizations of high value and, once successful, can cause huge economic losses, political impact, or even destruction of the target. For example, the Google Aurora (Aurora) attack (Wikipedia Contributors, 2024) in 2010 is a famous case of APT attack. In this attack, the attackers successfully infiltrated Google's search engine through careful planning, which lasted for several months and led to the leakage of the company's confidential information.

The low and slow nature of APTs makes it difficult to detect APTs using traditional network attack detection methods, which can only detect single-point attacks (Bridges et al., 2019; Singla et al., 2020),

and it is difficult to model the long-lasting behavior of an attack. Recent studies (Hossain et al., 2017, 2020; Milajerdi et al., 2019b,a; Hassan et al., 2019; Han et al., 2020; Hassan et al., 2020; Alsaheel et al., 2021; Wang et al., 2022; Chen et al., 2022; Xiong et al., 2020; Xu et al., 2022; Yan et al., 2022; Cheng et al., 2023a; Liu et al., 2022; Yang et al., 2023; Zengy et al., 2022; Rehman et al., 2024; Jia et al., 2023; Li and Chen, 2024) have shown that the provenance of the data is a promising approach to detect APTs and may become a more robust detection mechanism in the next generation.

Typically, data provenance transforms system logs into a provenance graph, which is a directed acyclic graph where nodes represent subjects (e.g., processes, threads, etc.) and objects (e.g., files, registries, network sockets, etc.) in the system, and edges denote the control-flow and data-flow relationships between these nodes. The provenance graph not only contains information about the activities in the system, but also incorporates temporal information, making it possible to have multiple edges between two nodes with different attributes (including time and specific operations, etc.). This design enables the provenance graph to comprehensively and accurately reflect the activity trajectory and attack path of APT attacks in the system. Compared with raw system audit data, provenance data have strong semantic representation and historical attack correlation capabilities. This makes

* Corresponding author.

E-mail address: gengrong@mail.neu.edu.cn (R. Geng).

the provenance graph an ideal threat modeling method to effectively describe and model the behavior and patterns of APT attacks.

Researchers have proposed many methods for APT attack detection based on provenance graphs. Earlier approaches (e.g., SLEUTH [Hossain et al., 2017](#), Morse [Hossain et al., 2020](#), HOLMES [Milajerdi et al., 2019a](#), CONAN [Xiong et al., 2020](#)) manually design various rules based on threat knowledge (e.g., kill chain model [Yadav and Rao, 2015](#), ATT&CK model [The MITRE Corporation, 2015-2024](#)) to detect APTs. Although they are effective, they still have the following limitations: the design of rules requires specialized domain knowledge; APT attacks often rely on zero-day vulnerabilities, and rule-based detection methods may not be effective in identifying unknown or new attack methods; rule-based methods have a large number of false positives, which increases the workload of analysts and leads to alert fatigue.

Machine learning-based analysis, on the other hand, does not require expert experience or knowledge and has demonstrated its effectiveness in automating APT analysis on provenance graphs ([Han et al., 2020](#); [Hassan et al., 2020](#); [Alsaheel et al., 2021](#); [Wang et al., 2022](#); [Chen et al., 2022](#); [Xu et al., 2022](#); [Yan et al., 2022](#); [Cheng et al., 2023a](#); [Liu et al., 2022](#); [Yang et al., 2023](#); [Zengy et al., 2022](#); [Rehman et al., 2024](#); [Jia et al., 2023](#); [Li and Chen, 2024](#)). However, there are still some challenges that constrain its application: (1) the lack of data (LOD) problem, learning-based approaches require a large amount of labeled data to train the machine learning models, while in practical situations the data of APT attacks are often difficult to obtain and label, which may lead to insufficiently trained or poorly generalized models; (2) the high consumption of computational resources, high arithmetic power requirements limit its practical application, and the computation time may also cause the real-time and efficiency of the detection system to be affected.

Based on the above problems, in this paper, we propose a self-supervised learning-based APT detection model, APT-MGL, which learns the embedding representations of nodes based on a new graph-masked self-encoder, thus transforming the detection problem into an outlier detection problem for malicious nodes without the need of real-world attack data to achieve node-level detection. Similar to previous works ([Wang et al., 2022](#); [Chen et al., 2022](#); [Xiong et al., 2020](#)), our model first extracts useful information from audit logs and constructs provenance graphs via regular expressions. Observing that the interaction objects and patterns of APT attack entities can be clearly distinguished from normal entities, removing the node types, we also consider the node's action information with the corresponding actuators and use word2vec to obtain their vector representations as node features. We perform a simple graph reduction scheme, considering that this results in the loss of statistical information, we collect the degree information of the nodes and fuse the features of these three dimensions through a multi-head self-attention mechanism. Then, self-supervised learning is used to combine graph features and structural information to obtain node embeddings. Our work references MAGIC ([Jia et al., 2023](#)) and uses improved GraphMAE2 ([Hou et al., 2023](#)) to learn the node embeddings. Finally, the computed embeddings are analyzed using an unsupervised outlier detection method to obtain the final detection results.

In summary, this paper makes the following contributions:

(1) We propose APT-MGL to detect malicious behaviors by modeling the behavioral patterns of normal network nodes, which consists of two key components. First, a masked graph self-encoder learns normal node behavioral patterns and embeds them into the vector space. Then, outlier detection acquires malicious behaviors that deviate from normal behavioral patterns.

(2) We describe the behavioral patterns of nodes in terms of node type, node behavior, and node degree, and use a multi-head self-attention mechanism for semantic enhancement.

(3) We employ a novel graph mask self-encoder to reconstruct node features and learn node neighborhood features from multiple perspectives to obtain a more comprehensive description of node behavior.

Combined with sample-based structural reconstruction, the output embedding contains both node neighborhood features and system state features.

(4) We evaluate our model on two widely used datasets. The evaluation results show that our model can effectively detect malicious activity-related nodes with higher precision and recall than existing apt detectors based on provenance graphs.

2. Previous works

Provenance-based detection methods have become a much-anticipated research area in detecting advanced persistent threats (APTs) attacks. The structure of the Directed Acyclic Diagram (DAG) of the provenance graph stores a large amount of information about the system activities, making it effective for complete causal analysis and flow computation. According to the different detection strategies, APT attack detection based on the provenance graph can be categorized into rule-based detection and learning-based detection.

Rule-based detection refers to the formulation of rule-based strategies based on known attacks. SLEUTH ([Hossain et al., 2017](#)) formulated label-to-label propagation rules for nodes to encode the assessment of trustworthiness and sensitivity of data and processes. However, due to the dependency explosion problem, which occurs when the number of dependencies between nodes grows exponentially in response to prolonged attacks, SLEUTH generates a large-scale graph in the face of prolonged attacks, which is not conducive to forensic analysis. Morse ([Hossain et al., 2020](#)) introduced label decay and label decline to mitigate the dependency explosion problem based on Sleuth. HOLMES ([Milajerdi et al., 2019a](#)) is a layered framework for APT detection that builds high-level scenario graphs for intrusion detection by mapping the behavior of the underlying entity to Tactics, Techniques and Procedures (TTPs) in the ATT&CK matrix. However, HOLMES is performed on the complete graph, which challenges the system memory. POIROT ([Milajerdi et al., 2019b](#)) manually constructs threat query graphs using APT attack reports and detects threats based on graph-aligned fuzzy matching provenance graphs. RapSheet ([Hassan et al., 2020](#)) introduces provenance graphs to commercial EDR for the first time and proposes Tactical Provenance Graphs (TPGs) to represent causal dependencies between EDR-generated threat alerts. CONAN ([Xiong et al., 2020](#)) introduces an innovative state-based detection framework, characterized by its feature of representing each process and file as a structure similar to a Finite-State Automaton (FSA). Such a design achieves a constant memory footprint. However, when faced with a high volume of concurrent operations in the system, such as numerous simultaneous file reading and writing tasks, CONAN fails to ensure real-time performance. In summary, although the rule-matching-based detection method achieves high detection accuracy as well as attack interpretability, it is difficult to achieve the desired performance with predefined rules for unknown attacks, and the formulation of rules requires expert experience and expertise.

Learning-based detection involves training a model to recognize characteristic patterns of APT attacks. For example, anomalous behavior detection methods build profiles of normal activities and determine violations of normal activities as anomalies. StreamSpot ([Manzoor et al., 2016](#)) captures the dynamics of network behaviors by constructing heterogeneous stream graphs and identifies potential network attacks through cluster-based anomaly detection methods. UNICORN ([Han et al., 2020](#)) uses graph sketching techniques that can analyze provenance graphs that contain rich contextual and historical information in long-running systems to identify unknown slow attacks. Due to the limitations of graph kernel methods, they have difficulties in detecting stealthy threats. PROGRAPHER ([Yang et al., 2023](#)) combines full graph embedding (via graph2vec) and sequence learning (via TextRCNN) to analyze snapshots of the source graph, effectively learning a representation of normal system behavior. As a graph-level method, PROGRAPHER shares the same limitations as StreamSpot and Unicorn.

SHADEWATCHER (Zengy et al., 2022) maps the security concept of system-entity interactions to the recommendation concept of user-item interactions and identifies cyber threats by predicting the system entity's preferences for the entities it interacts with. ATLAS (Alsaheel et al., 2021) proposes a sequence-based attack investigation learning approach to establish critical patterns of attack and non-attack behavior from provenance graph diagrams. ATLAS requires providing some known malicious entities as the starting point of the path. APT-KGL (Chen et al., 2022) uses graph embedding techniques to learn the vector representation of each node of a heterogeneous graph and proposes a local graph sampling approach to solve the incremental learning problem. THREATTRACE (Wang et al., 2022) proposes a multilayer model training approach by building a multilayer graph neural network model, which implements further use of submodels to extract implicit node labels. Deepro (Yan et al., 2022) allows for a nuanced identification of APT attacks by recognizing related processes. However, the application of supervised learning presents difficulties due to the significant sample disparity between benign and attack instances. DepComm (Xu et al., 2022) compartmentalizes a broad source graph into communities centered around processes. Each community then generates a representative InfoPath summarizing its own characteristics. Flash (Rehman et al., 2024) uses the same strategy as THREATTRACE, except that Flash captures more semantic attributes as node features. MAGIC (Jia et al., 2023) employs a novel graph masking self-encoder that forces the model to capture and understand the intrinsic structure and patterns of the graph data by randomly masking some of the node features or structures to effectively reconstruct the masked information. MAGIC uses node types as the reconstructed node features and ignores other meaningful information.

In this paper, we propose a self-supervised learning-based APT detection model to learn the embedding representation of nodes based on a new graph-masked self-encoder, thus transforming the detection problem into an outlier detection problem for malicious nodes and realizing node-level detection. Our model performs entity embedding in terms of node types, interaction behaviors, and distribution of the number of interaction types, and employs a multi-head self-attention mechanism for semantic fusion. In addition, our model attempts to reconstruct the structure of the provenance graph, resulting in a more comprehensive understanding of the semantics of the entity and demonstrating better detection performance. Compared with rule-based and supervised learning approaches, our model shows greater potential to identify zero-day vulnerabilities used in APT campaigns and achieves higher accuracy and recall metrics.

3. Overview

3.1. Motivation example

The provenance graph explicitly represents the control flow and data flow between system entities, which records all events in the system and the causal relationships between them, demonstrating rich semantics and powerful expressiveness. With the provenance graph, the causal relationships between the logs can be traced and detailed information about the execution history of the system can be preserved, which facilitates the detection of long-term and stealthy APT attacks. Fig. 1 shows a provenance graph of DARPA TC # 3 Cadets (Anon, 2023a), which utilizes an *Nginx* vulnerability for a *SSH* injection attack. First, the attacker (81.49.200.166) establishes a connection with the susceptible *Nginx* server to procure a shell. Utilizing this shell, the assault successfully pulls in and places a harmful payload in */tmp/vUgefai*, subsequently executing it with root-level permissions. The elevated process *vUgefai* then attempts to move laterally to 61.167.39.128. The attacker further plans to inject a process like *SSH* with persistent access and long-term listening, but fails.

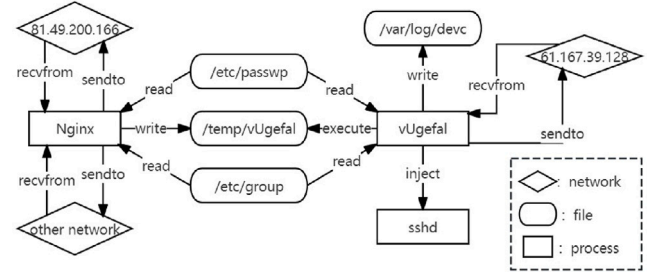


Fig. 1. The provenance graph of a real-world APT attack.

3.1.1. Limitations of existing techniques

Complex feature design and reliance on expert knowledge: Early works such as Hossain et al. (2017, 2020), Milajerdi et al. (2019a) rely on expert-designed features, which not only require a large amount of domain knowledge but also tend to fail when dealing with new types of attacks and cannot flexibly adapt to changes in attack behavior.

Difficulty in capturing dependencies: Statistics-based approaches (Hassan et al., 2019; Wang et al., 2020) assign anomaly scores to system entities or interactions by identifying their scarcity and use this to detect APT signals. However, statistics-based approaches can only detect the scarcity or anomaly of specific behaviors, but it is difficult to globally understand the causal chain between these behaviors. For example, in our motivation process *vUgefai* reads sensitive files */etc/passwd* and writes logs */var/log/devc*, which are normal behaviors in some scenarios, but when combined with its communication with a remote network node 61.167.39.128, we can deduce that it performs malicious behaviors.

High consumption of computational resources: Many existing methods (Alsaheel et al., 2021; Chen et al., 2022; Zengy et al., 2022) use complex feature extraction and analysis mechanisms to improve the detection rate, leading to a very high consumption of computational resources and makes these systems difficult to effectively deploy in real-world applications.

3.1.2. Intuition of our approach

In this paper, our goal is to identify suspicious nodes in host systems caused by intrusions. Our core idea is that, in the source graph, attack patterns exhibit specific attack characteristics that make the attacker's behavior different from normal activities. Compared with nodes representing benign activities, malicious nodes present uniqueness in local structure or contextual information due to the purpose of their actions. For this reason, APT-MGL introduces a masked graph self-encoder to efficiently obtain high-quality node embeddings through self-supervision while avoiding excessive resource consumption. In addition, APT-MGL uses a multi-head self-attention mechanism to further enhance the model's ability to perceive behavioral dependencies across time periods and between system entities.

3.2. System architecture

Our model architecture is shown in Fig. 2, which consists of four parts: (1) log preprocessing, (2) node feature collection and embedding, (3) node feature fusion and model training, and (4) node clustering and outlier detection.

(1) Log preprocessing: Provenance data (e.g., system events such as file operations, network accesses, etc.) can be collected from kernel-level information by using an operating system auditing module (e.g., windows ETW Microsoft Learn, 2024), which consists of logs of host events, such as process execution, file operations, and network connections. The log preprocessing module converts raw logs from different operating systems into static provenance graphs and extracts system entities and their interactions into nodes and edges, respectively. In

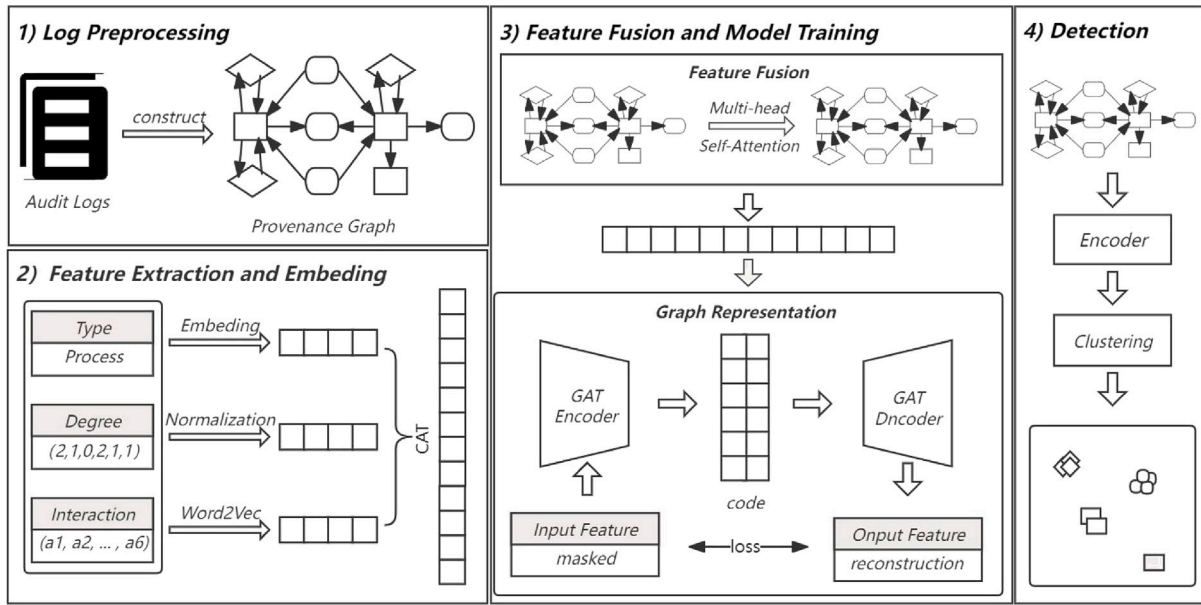


Fig. 2. Architecture of detection model.

addition, a simple strategy is used to compress the raw provenance graph to remove redundancy.

(2) Collection and embedding of node features: We consider three dimensions of node features, i.e., node types, node behavioral features, and node degree features. Specifically, we collect its behavioral records for each node and then use word2vec (Mikolov et al., 2013) to train a vector of these interaction behaviors. These features are transformed (embedded) and then feature-enhanced (embedded in the training parameters in the training model) through a multi-head self-attention mechanism, which serves as the initial features for GNN model training.

(3) Node feature fusion and model training: For the initial features obtained after word2vec training in (2), the multi-head self-attention mechanism is first used to obtain the weights of each type of features, and then the fused features are fed into the graph neural network to learn the contextual information of the nodes. We employ a novel graph mask self-encoder, GraphMAE2 (Hou et al., 2023), and a graph attention neural network (GAT) (Vaswani et al., 2017) as the encoder and decoder of the model, which combines structural reconstruction based on samples to embed, propagate, and aggregate edge and node attributes into the output embedding, which contains both node embeddings and system state embeddings. Only benign audit logs are used for training in the training phase to model benign system behavior.

(4) Node clustering and outlier detection: Using the encoder part of GraphMAE2, the graph mask self-encoder in (3), as a node vector generator, the output embedding-based outlier detection method is used to detect outliers in the system behavior.

3.3. Threat model

The threat model in this paper is based on previous provenance work on APT detection (Wang et al., 2022; Chen et al., 2022; Yan et al., 2022) and assumes that the auditing modules of the system (e.g. Auditd Die.net, 2020, ETW Microsoft Learn, 2024) are equipped to thoroughly document crucial system interplays such as file manipulations, network accesses, and others, at the system kernel level. With the foundational operating system and the system audit module collectively constituting the chief portion of the Trusted Computing Base (TCB), our assumption rests on a belief that these components are immune to assault from the kernel plane. In addition, this paper further assumes that the integrity of the data provenance graphs used for the analysis is safeguarded from potential attackers.

Table 1

Entities and Causality Relationships.

Subject	Object	Relationship
Process	Process	fork, execute, exit, clone....
	File	read, open, close, write, rename, mmap, update...
	Network	connect, sendto, recvfrom, accept...

4. Design

4.1. Provenance graph construction and preprocessing

For system audit logs from different OS log auditing tools, our provenance graph-based APT detector first converts them into a provenance graph. APT-MGL interprets each audit trail event as a link wherein the origin node exemplifies the event's main subject (i.e., the process), the target node signifies the operation's target (i.e., file and network), and the connecting links illustrate the causal relationships amongst these entities. As in previous work (e.g. APT-KGL Chen et al., 2022, Deepro Yan et al., 2022, MAGIC Jia et al., 2023), only processes, files, networks (network, inter-process communication (IPC), sockets, etc.). MemoryObjects and UnnamedPipeObjects related entities are considered (note: MemoryObjects and UnnamedPipeObjects are not detected as malicious nodes even if they are involved in malicious activities because these types of nodes are frequently operated and accessed during normal system operations, and marking them as malicious nodes may lead to a large number of false positives, making it difficult for the detection system to distinguish between normal system behavior and malicious APT activities, and this type of node is difficult to label accurately). Table 1 shows the entity types considered with some of the associated edges.

A provenance graph encompasses an extensive count of surplus and attack-indifferent nodes and edges, stemming from its attempt to encapsulate every causal relationship amid system components. This contributes to the magnitude of the origin graph, thereby hindering efficient model training. As illustrated in Fig. 3, our approach is centered around the unification of all repetitive edges within the origin graph, a strategy that is relatively straightforward. Duplicate edges indicate repeated operations between nodes, and previous studies (Alsaheel et al., 2021) have shown that these duplicate edges do not provide additional useful information for attack analysis. Unlike existing work

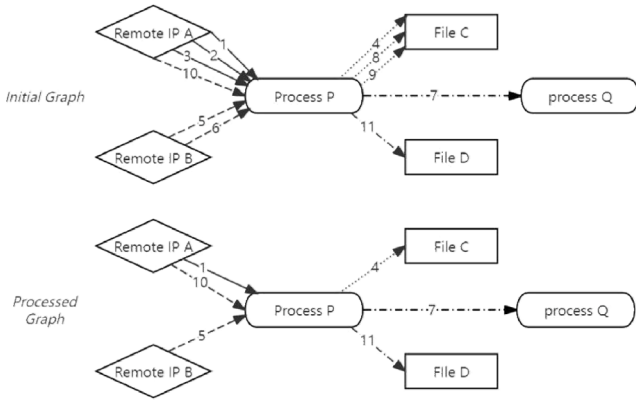


Fig. 3. Merge duplicate edges. The arrows represent different types of edges.

on provenance graph parsimony (Hossain et al., 2018; Mikolov et al., 2013), we do not attempt to merge nodes with the same interaction behavior or remove edge nodes (e.g., read-only files), as they are also part of the uniformly benign behavior, and merging these nodes may distort the GNN's embedded representations of these nodes, leading to a biased understanding of the nodes by the type.

4.2. Node embedding

The performance of a graph masked self-encoder architecture in processing graph data largely depends on its ability to represent the chosen node features and graph structure. Highly discriminative features better characterize the uniqueness of nodes and their relationships in the graph. We consider three key aspects of node properties:

- **Node Types:** This fundamental attribute indicates the basic behavioral patterns in the provenance graph, as different types of nodes interact differently with each other.
- **Node Degree Features:** This includes both node in-degree and out-degree, which record the frequency of interactions between nodes. Due to the graph compression strategy, some node statistical information may be lost, potentially leading to the omission of certain anomalous behaviors. To address this, we collect the original node degree parameters to preserve relevant statistical information.
- **Node Behavioral Characteristics:** This refers to the interaction actions that nodes take with other nodes. Analysis of system audit logs reveals that some nodes exhibit fixed behavioral patterns in their interactions. These characteristics need to be encoded into a vector space for the model to utilize effectively.

Node types: As the base attributes of nodes, node types can be obtained while constructing the provenance graph without additional collection. Usually, node types can be used as input vectors for model training after simple one-hot encoding. Considering that one-hot encoding is sparse, which is not conducive to the learning and optimization of neural networks, embedding layers are used to obtain dense vector representations.

Nodal Degree Characterization: Interactions between entities in the network (e.g., users, devices, IP addresses, services, etc.) can reveal specific behavioral patterns. By analyzing the type and frequency of these interactions, the difference between normal and abnormal behavior can be identified. For example, a user who suddenly establishes connections with a large number of unrelated external IP addresses may be an indication of malicious activity. The distribution of the number of entity interaction types can provide a relatively rich feature set, and different interaction types (e.g., HTTP requests, SSH connections, file transfers, etc.) can reflect different aspects of network activities,

thus providing more contextual information for the model. In addition, various node types engage in distinct interactions; for instance, file read and write operations occur exclusively between processes and files, rather than between sockets. This variation in interaction type distribution can help illustrate the diversity of entity types (Luo et al., 2024).

To characterize the distribution of the number of such interaction types, we collect the in- and out-degrees of nodes to different types of edges, characteristics that also preserve the statistics of the provenance graph. First, calculate the number of different interaction types M and set the type mapping function of the edges to $M_e : \Sigma \rightarrow N$, which maps the interaction type types to the integer range from 0 to $N-1$. Then, we learn a mapping function $L : V \rightarrow N^{2 \times M}$ to assign the characteristics of $\forall v \in V$ as $L(v) = [a_0, a_1, \dots, a_{M-1}, a_M, a_{M+1}, \dots, a_{2M-1}]$, where a_i is indicated as Eq. (1):

$$a_i = \begin{cases} |\{e \in \text{In}(v), M_e(\text{edgeType}(e)) = i\}|, & i \in 0, \dots, M-1 \\ |\{e \in \text{Out}(v), M_e(\text{edgeType}(e)) + M = i\}|, & i \in M, \dots, 2 \times M-1 \end{cases} \quad (1)$$

Node Behavioral Characteristics: Usually, different types of nodes exhibit their specific functions and thus show fixed behavioral characteristics in the provenance graph. When a node is attacked, the attacker's behavior of using the node for malicious activities will cause the node to exhibit entity behaviors that are different from normal behaviors. We collect the behavioral history of nodes from the original provenance graph and transform these attributes into a dense vector space using the Word2Vec (Mikolov et al., 2013) model. Word2vec learns semantic relationships between words by training a neural network model that converts words into numeric vector representations, such that words with similar semantics are close together in the vector space. Specifically, we collect interaction sequences between nodes and their 1-hop neighbors to form summary sentences for each node. System events are sorted by timestamp to maintain temporal order. Each sentence is then encoded into a fixed-length vector by a Word2Vec model trained on benign system logs.

Moreover, since the word2vec model ignores temporal and causal order, this may lead to a misjudgment in threat identification as it fails to capture the actual order in which malicious activities occur. To address this issue, we draw on the work of Flash (Rehman et al., 2024) and introduce a temporal encoding scheme. In this scheme, individual weights are allocated to every word-embedding. Throughout the sentence, these weights are accumulated to form an indicative representation of the embedding that conveys temporal information. This scheme references the notion of positional encoding in Transformer (Vaswani et al., 2017) and aims to convey information about the position of each marker in the sequence. Specifically, the position-encoded vector of tokens at position i is computed by a specific mathematical operation that efficiently captures and represents the temporal ordering information in the sequence, as shown in Eq. (2):

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \\ PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2)$$

where d denotes the dimension of the embedding and j denotes the index of the dimension within the embedding. The position encoding vectors are then added to the Word2Vec embedding vectors for each marker at that position.

Feature fusion: After obtaining the three aspects of node features (node type, behavior features, and node degree features), we use a multi-head self-attention mechanism (Devlin et al., 2018) to perform feature fusion, which results in higher quality node embedding representations. Specifically, the input data (original features) is divided into multiple parts, each processed by an independent "head". Each "head"

focuses on different semantic information when processing the data, thereby capturing various features present in the data.

The core idea of the multi-head self-attention mechanism is to learn multiple types of feature or dependencies in the data through parallel “heads”. Each “head” calculates its own unique attention weight and focuses on different information dimensions, which can more comprehensively understand the relationship between nodes and the details in the graph structure. Specifically, the calculation formula for each “head” is shown in Eq. (3), which shows how to generate the attention representation of each head through a weighted and aggregated mechanism.

These independently calculated attention representations are then spliced together to form a richer feature representation. Finally, these spliced representations are mapped through a linear transformation layer to generate the final node embedding. This processing approach can improve the model’s expressiveness and flexibility for graph data, enabling it to capture complex patterns and relationships more effectively.

$$\text{head}_i = \text{softmax} \left(\frac{q_i k_i^T}{\sqrt{d/n}} \right) v_i \quad (3)$$

$$\text{MultiHead}(Q, K, V) = \text{concat}(\text{head}_1, \dots, \text{head}_n) W$$

4.3. Masked graph autoencoder

The model training phase employs a self-supervised learning approach using graph mask self-encoding techniques to generate node embeddings. Our dataset consists of normal provenance graph data devoid of malicious nodes, with the goal of developing effective representation models by learning patterns from these normal behaviors. We have selected GraphMAE2, an advanced iteration of GraphMAE (Hou et al., 2022), as our primary learning mechanism.

GraphMAE2 improves upon its predecessor by incorporating a graph masking technique. During training, the model randomly masks portions of nodes and edges. This strategy requires the model to predict and reconstruct the obscured elements, thereby learning intricate relationships and dependencies within the graph. By operating in a self-supervised manner, GraphMAE2 fully exploits large-scale unlabeled data to derive meaningful node embeddings, without relying on additional labeled datasets. Its sophisticated masking and reconstruction methods enhance the model’s ability to handle diverse graph structures and produce robust, accurate embeddings.

These embeddings effectively capture the graph’s underlying structure and node interactions, which provides a solid foundation for tasks such as node classification and link prediction. This approach enables us to leverage normal data for model training, ensuring that the learned representations are well-suited for detecting anomalies and comprehending normal behaviors.

4.3.1. Node masking process

Masking the nodes before training the graph representation module encourages the model to learn how to recover or predict the features of the masked nodes from the remaining graph structure, which is the same step as the process of GraphMAE. First, a certain percentage of nodes from the input graph are randomly selected as candidate-masked nodes. In GNN, each node relies on its neighboring nodes to enhance/recover its features (Gilmer et al., 2017). Uniformly distributed random sampling helps prevent potential bias centers, where the neighbors of a node are not masked or visible (Hou et al., 2023). For selected nodes to be masked, that is, the features or representations of these nodes will be set to a particular masking token (e.g. an all-zero vector) or a noise vector, which manifests itself by replacing their initial embedding values with a special mask token X_{mask} , thus completely masking any original feature information of these nodes. Furthermore, although the nodes are masked, their connectivity (i.e., edges) to other

nodes in the graph is preserved, i.e., the complete graph structure is retained. In this way, the model can still make use of the neighbor node features of these nodes when predicting the features of the masked nodes.

Given the initial embedding value X_n of a node, we will mask the node during the training process according to the following steps :

$$\text{emb}_n = \begin{cases} x_n, & n \notin \tilde{N} \\ x_{\text{mask}}, & n \in \tilde{N} \end{cases} \quad (4)$$

where $\tilde{N}$ is the randomly selected masked node and emb_n is the embedding of node N ready to train the graph representation module.

4.3.2. Graph encoder

According to the experimental results in GraphMAE/GraphMAE2, the use of multiple stacking graph attention network (GAT) layers has the best embedding results. Unlike GCN (Kipf and Welling, 2016), GraphMAGE (Hamilton et al., 2017), and other GNNs, GAT (Veličković et al., 2017) is able to adaptively learn the weights between different nodes by introducing an attention mechanism to more accurately capture the dependencies between the nodes. By stacking multiple GAT layers to construct a deep model, more complex graph structure information can be captured. The information conveyed by each GAT layer is represented as follows, which contains node features with the corresponding edge features, where emb_e is the initial edge embedding:

$$\text{Message}(src, dst) = W_{msg}^T (h_{src} + \text{emb}_e) \quad (5)$$

GAT then uses the attention mechanism to compute the correlation or importance between nodes. Specifically, for each node, GAT calculates the attention score between it and all its neighboring nodes and computes the attention coefficient:

$$\alpha(src, dst) = \text{LeakyReLU}(W_{ad}^T h_{dst} + W_{am} \text{Message}(src, dst)) \quad (6)$$

$$a_{dst} = \text{Softmax}(\alpha(src, dst))_{dst}$$

After obtaining the attention coefficients $\alpha(src, dst)$, GAT can aggregate the features of the neighboring nodes of the current node based on these coefficients. Specifically, GAT will linearly transform the features of neighboring nodes (via the weight matrix W , as shown in Eq. (5)) then multiply them by the corresponding attention coefficients, and finally sum them up to get the new feature representation of node n :

$$\text{AGG}(h_{dst}, h_n) = W_{self} h_{dst} + \sum_{i \in N_n} a(i, dst) \text{MSG}(i, dst) \quad (7)$$

$$h_n^l = \text{AGG}^l(h_n^{l-1}, h_{N_n}^{l-1})$$

where h_n^l is the hidden embedding l of node n in the first GAT layer, h_n^{l-1} is the hidden embedding $l-1$ of the first layer and N_n is the neighborhood of one hop of n .

For the intermediate layers of multilayer graph neural networks, which capture the complex relationships between nodes in the graph data, the representation of nodes can be enhanced by stacking the output of multiple GAT layers, which can progressively learn and aggregate information from their neighbors. The final node embedding is as follows:

$$h_n = \text{emb}_n \parallel h_n^1 \parallel \dots \parallel h_n^l \quad (8)$$

4.3.3. Graph decoder

In the Encoder-Decoder model, the role of the encoder is to convert input data (e.g. nodes, edges, and features in graph data) into one or more representations (or embeddings) that capture key information about the input data. These node embeddings are typically used as the output of the representation module, which is then passed to a decoder

(Decoder). The Decoding process transforms the potential representations (or embeddings) generated by the encoder back into predictions of the original graph structure or node features, thus enabling feature reconstruction, a process that aims to improve the performance of graph Self-Supervised Learning (SSL) by normalizing feature reconstruction. In addition, we retain the design of graph structure reconstruction, and the final embedding integrates masked feature reconstruction with sample-based structure reconstruction.

Multi-View Toochastic Rmasking Decoding: A new stochastic masking is performed in the decoding phase, which is used to introduce randomness in the reconstruction of the feature space in order to reduce the overfitting to the input features and to improve the robustness of the model. That is, unlike the fixed remasking strategy in GraphMAE, nodes are equally selected for remasking regardless of whether they were masked before embedding:

$$\tilde{h}_i = \begin{cases} h_{[M]} & \text{if } v_i \in \bar{V} \\ h_i & \text{if } v_i \notin \bar{V} \end{cases} \quad (9)$$

where $\bar{V}$ is a randomly sampled subset from the node set V . The decoder then reconstructs the input from the remasked embedded features $\tilde{H}$. The process performs several random remasking operations and tries to recover the original input features from these masked (or partially masked) embeddings. This process is equivalent to decoding and reconstructing the input features from different 'views' or 'partial views', thus increasing the adaptability of the model to changes in the input features. Finally, the reconstruction error is measured using the scaled cosine error and summed over the errors of the training views:

$$\mathcal{L}_{\text{input}} = \frac{1}{|\bar{V}|} \sum_{j=1}^K \sum_{v_i \in \bar{V}} \left(1 - \frac{x_i^T z_i^{(j)}}{\|x_i\| \cdot \|z_i^{(j)}\|} \right)^{\gamma} \quad (10)$$

Latent Representation Prediction: In SSL (Self-Supervised Learning), Latent Representation Prediction is a generic tool whose main objective is to improve the model's understanding and generalization of data by training it to predict the latent representations (or embeddings) of the features. The focus of this section is to construct an additional informative prediction goal, a strategy that requires the model to not only reconstruct input features from the masked node embeddings, but also to predict potential representations in the embedding space.

A GAT network serves as a target generator to derive potential prediction targets from the unmasked graph. The process involves initially passing the original graph through the target generator to create the target representation, followed by projecting the encoded results of the masked graph into the representation space to obtain the potential predictions:

$$\hat{\mathbf{Z}} = g(\mathbf{H}; \theta), \hat{\mathbf{X}} = g'(f'_E(\mathbf{A}, \mathbf{X}; \xi); \xi) \quad (11)$$

where $g(\cdot; \theta)$ is the projection matrix, $\mathbf{H}$ denotes the original node feature matrix, $f'_E(\mathbf{A}, \mathbf{X}; \xi)$ is the node embedding generated by the encoder, θ and ξ denotes the learnable weights. $\hat{\mathbf{Z}}$ and $\hat{\mathbf{X}}$ are the target generator and encoder representations in the embedding space, respectively.

The encoder and target generator share the same architecture but utilize distinct weights. Both are trained to synchronize their outputs at the target node by minimizing a scaled cosine error via gradient descent, optimizing the encoder and projection matrix parameters:

$$\mathcal{L}_{\text{latent}} = \frac{1}{N} \sum_i \left(1 - \frac{\tilde{z}_i^T \tilde{x}_i}{\|\tilde{z}\| \cdot \|\tilde{x}\|} \right)^{\gamma} \quad (12)$$

While the parameters of the target generator are updated by using an exponential moving average of the weights decaying r :

$$\xi \leftarrow \tau \xi + (1 - \tau) \theta \quad (13)$$

Graph Structure Reconstruction: Link prediction is a common means of detecting APT attacks in provenance graphs (Luo et al., 2024; Cheng et al., 2023b), where anomalous behaviors that are significantly different from the normal pattern can be identified by predicting missing links or evaluating the weights of existing links in the graph. We only predict link relationships between different nodes and do not attempt to predict link types due to the inherent discrepancy between the discrete, sparse nature of graph edges and the continuous, high-dimensional attributes of nodes (Shi et al., 2023). Such disparity hampers the simultaneous reconstruction of both edges and node features. Specifically, we randomly select a certain number of node links as positive samples for structural reconstruction and generate non-existing links for them as negative samples, and then predict their labels using a simple MLP layer:

$$\text{prob}(dst, src) = \sigma(\text{MLP}(h_{src} || h_{dst})) \quad (14)$$

TRAINING: The flow of learning graph embedding representation is shown in Fig. 4. All losses in the training process are combined through a mixing factor:

$$\mathcal{L} = \mathcal{L}_{\text{input}} + \beta \mathcal{L}_{\text{sr}} + \lambda \mathcal{L}_{\text{latent}} \quad (15)$$

4.4. Detection

We utilize embeddings generated by a graph representation learning module for APT detection, using an unsupervised outlier detection approach. The goal of this detection model is to identify malicious system entities or states in the presence of only a priori knowledge of benign system behavior. By analyzing the embeddings generated by graph representation learning, we can find that entities with similar interaction behaviors tend to form clusters. Outliers in the system state embeddings, on the other hand, represent uncommon and suspicious system behaviors.

We tried several outlier detection algorithms and chose the K Nearest-Neighbor (KNN) algorithm to perform the anomaly detection task. In the training process, the embedding representation of benign output is first extracted from the training source graph to fit the KNN model representation. Then, for the target embedding, KNN first searches for its nearest neighbors K and obtains its anomaly score by calculation of similarity. If the anomaly score exceeds a predetermined threshold θ , the target embedding is identified as anomalous, suggesting that the associated system entity or state could be malicious. The following is the workflow for the detection module:

$$\begin{aligned} \mathcal{N}_x &= \text{KNN}(x) \\ \text{AvgDist}_x &= \frac{1}{|\mathcal{N}_x|} \sum_{x_i \in \mathcal{N}_x} \|x - x_i\| \\ \text{score}_x &= \frac{\text{AvgDist}_x}{\text{AvgDist}_{\text{normal}}} \\ \text{Detect}_x &= \begin{cases} \text{abnormal}, & \text{score}_x \geq \theta \\ \text{normal}, & \text{score}_x < \theta \end{cases} \end{aligned} \quad (16)$$

4.5. Model adaptation

As the behavior of the system changes, APT-MGL can raise false alarms about benign entity interactions of the system that were not observed during the training phase, which is known as the concept drift problem. Recent studies have kept up with the evolution of system behavior by forgetting outdated information (Han et al., 2020) or implementing model adaptation mechanisms (Singla et al., 2020).

APT-MGL implements network behavior modeling through self-supervised learning, which means it can adjust model parameters using

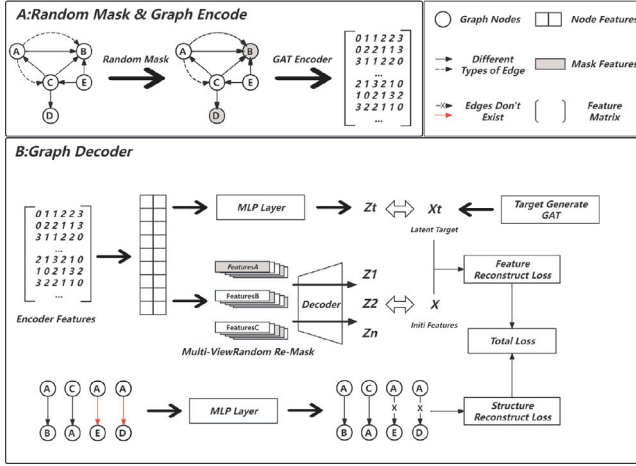


Fig. 4. Graph Embedding Training Process.

any normal behavior. We implement a model adaptation regime where the system receives new data at runtime, and the self-supervised model detects these new data based on learned patterns and determines whether they are consistent with previous normal behavior. If the new data are considered “normal” (i.e., not anomalous behavior) or are analyzed and considered “normal” (i.e., benign feedback), they are used for parameter tuning to help the model gradually adapt to changes in the environment.

One issue to consider is that model tuning based on the latest data can pose a challenge to the system’s resource consumption. Fortunately, APT-MGL does not require excessive computational resources and is able to detect and retrain the model before the next batch of data arrives. We will discuss the runtime consumption of APT-MGL in Section 5.5.

5. Evaluation

5.1. Datasets and experimental setup

We evaluated our model on two open source datasets, the DARPA TC-E3 dataset (Anon, 2023a) with the StreamSpot dataset (Anon, 2023b). As shown in Table 2, the DARPA TC-E3 dataset is from the Transparent Computing (TC) project of the Defense Advanced Research Projects Agency (DARPA). The project organizes red and blue teams to conduct attack and defense exercises, during which fine-grained behavioral data is collected, attacks are detected and forensically traced, and a report is generated. The red team utilizes different vulnerabilities to carry out APT attacks and benign activities on hosts with multiple operating systems. Blue expects to identify these attacks by augmenting the fine-grained behavior of the system on the terminal with terminal-based data collection and analysis. The source data and groundtruth are publicly available, and by analyzing the groundtruth file provided by the official website, we can pick the names of the entities related to the attacks and then go through the entity names to match the uid in the original json file to include all the nodes involved as much as possible. We use the labels provided by Threatrace (Wang et al., 2022) and divide them by time, where the first 80% of the data is used for model training and the rest is used to test model performance.

The StreamSpot dataset, shown in Table 3, contains a total of six scenarios, five of which are normal behaviors, including watching YouTube videos, detecting emails, etc., as well as one attack behavior.

Table 2
OVERVIEW OF DARPA DATASET.

DatasetScene	# of benign nodes	# of abnormal nodes	# of edges
THEIA	1,598,647	25,319	2,874,821
Trace	3,220,594	68,082	4,080,457
CADETS	1,614,189	12,846	3,303,264

Table 3
OVERVIEW OF STREAMSPOT DATASET.

Scene	# of graphs	Average # of nodes	Average # of edges
Benign	500	8315	173,857
Attack	100	8891	28,423

The attack scenario is downloading a program from a malicious URL and exploiting a flash memory vulnerability to gain system administrator privileges. Each scenario was run 100 times during the collection of this dataset and the information was logged using SystemTap. Since this dataset does not provide specific malicious behavior annotations, we perform graph-level detection on this dataset, i.e., average pooling of all node embeddings as a graph-level representation. Similarly, 80% of the normal behavior graph is selected as the training set and the remaining is used for detection along with malicious behavior.

We conducted our experiments on a Windows test server equipped with an 8-core 64G100GXSKY-GPU + 80GGPU specification. Our model is implemented in Python 3.11 with about 3.5K lines of code, using the Networkx graph processing library to build & store the provenance graph, PyTorch and DGL for graph encoding, and Scikit-learn for the detection module.

For the hyperparameters of the model, the scale factor γ in the loss of feature reconstruction is set at 3, the number of neighbors k is set at 10, the learning rate is set to 0.001, the weight decay factor is 5×10^{-4} , the number of graph encoder layers and the number of multiview rerandomization masks are both set to 3, the mask rate is set to 0.5, the weight of structural reconstruction of sorts is set to 1, and the weight of potential representation prediction loss is set to 10.

Precision, Recall, and F1-score are used to evaluate the effectiveness of the model in detecting APT attack nodes. Precision denotes the percentage of anomalous nodes associated with APT activities that are successfully predicted by the model. Recall, on the other hand, represents the proportion of all nodes associated with APT activities that the model successfully detects. F1-score, on the other hand, is the reconciled average of Precision and Recall, providing a balanced evaluation metric.

5.2. Comparison experiments

We validate the experimental results of our model on each of the above datasets and compare them with the following existing detection models: MAGIC (Jia et al., 2023), Threatrace (Wang et al., 2022), ThreatSniffer (Luo et al., 2024), Unicon (Han et al., 2020), and StreamSpot (Anon, 2023b). These models represent different approaches in the field of APT detection, covering a wide range of scenarios such as unsupervised, semi-supervised, and streaming data processing, and are widely used in the open literature and are representative. Specifically:

- MAGIC is an unsupervised detection model that learns to detect APT behaviors using masked graph representations, does not rely on labeled data, and is suitable for unlabeled environments.
- Threatrace is a semi-supervised model that uses normal data for traceability analysis, identifies APT attack paths by comparing them with historical normal behavior, and performs anomaly detection;

Table 4
Detection Results on Different Datasets.

Dataset	Ground Truth		#TN	#FN	#TP	#FP	Precision	Recall	FPR	F1-Score	AUC
	#Benign	#Malicious									
StreamSpot	100	100	100	0.59	100	0.0	99.41%	100%	0.59%	99.71%	99.95%
DARPA E3 CADETS	344 327	12 846	344 132	32	12 814	195	98.50%	99.75%	0.056%	99.12%	99.93%
DARPA E3 THEIA	319 475	25 319	319 288	30	25 289	160	99.37%	99.88%	0.05%	99.63%	99.95%
DARPAE3 Trace	616 021	68 086	615 842	17	68 069	179	99.74%	99.97%	0.029%	99.86%	99.97%

Table 5
Comparison with Other Semi-Supervised APT Detection Methods on Different Datasets.

Dataset	Approach	Precision	F1-Score	Recall	FPR
StreamSpot	StreamSpot	73%	81%	91%	6.6%
	Unicon	95%	96%	93%	1.6%
	Threatrace	98%	99%	99%	0.4%
	MAGIC	99%	99%	100%	0.6%
	Ours	99%	99%	100%	0.59%
DARPA E3 CADETS	Threatrace	90%	95%	99%	0.2%
	MAGIC	93.7%	96.5%	99.75%	0.2%
	Ours	98.5%	99.12%	99.75%	0.056%
DARPA E3 THEIA	Threatrace	87%	93%	99%	0.1%
	MAGIC	98.23%	99.10%	99.99%	0.14%
	ThreatSniffer	74%	85%	100%	N/A
	Ours	99.37%	99.63%	99.88%	0.05%
DARPA E3 TRACE	Threatrace	90%	95%	99%	0.2%
	MAGIC	98.75%	99.36%	99.97%	0.1%
	Ours	99.74%	99.86%	99.97%	0.029%

- ThreatSniffer combines causal reasoning and machine learning to detect abnormal behavior that deviates from normal patterns by analyzing the causal relationship of normal behavior;
- Unicon performs semi-supervised APT detection through graph embedding and clustering for a wide range of cyber threat scenarios.
- StreamSpot is a semi-supervised model designed for streaming data, focusing on real-time APT detection in large-scale networks.

These models are chosen for comparison because they represent different technology paths and application scenarios for APT detection, allowing us to comprehensively evaluate the performance of the models in APT detection.

We first show the detection results of our model on each dataset and then compare them with existing APT detection methods.

Table 4 shows the experimental results of our model on different datasets, and the results show that the model can accurately detect APT attacks in different experimental scenarios and achieve high precision and recall.

Our model shows stunning detection results on the StreamSpot dataset because the StreamSpot dataset only collects a single user activity for each scenario, and the behavioral models of the nodes are fully presented in the provenance graph without any other interference. By learning different behavioral models, they can be easily separated, which justifies our approach of extracting node features.

In the DARPA data set, our model also achieves extremely high performance, reaching an average precision of 99.53% with 99.86% recall. This indicates that even in more complex scenarios, our model is still able to learn the behavioral patterns of the nodes and distinguish them. We analyze false-positive and false-negative points in different scenarios and find that for different systems these misclassified nodes are of different types, for example, the false-positive points in the CADETS data are mostly *FileObject* nodes, whereas the false-positive points in the TRACE data are mostly *NetworkObject* nodes. However, Process nodes are rarely misclassified, indicating that our model is good at discovering entities with active behavior. Typically, malicious activities are performed by Process nodes in conjunction with their child nodes and associated resource nodes (Xu et al., 2022), and by detecting maliciously behaving Process nodes, security practitioners

can quickly locate the resource nodes they are exploiting in order to recover the complete attack activity.

Experiments were conducted using the same data with the existing detection models to compare the results of our models. Unicon and StreamSpot are graph-level detectors and we conducted experiments only on the StreamSpot dataset. Threatrace and MAGIC both provide multi-level detection and we conduct complete experiments on them and compare them. ThreatSniffer only detects process nodes, and to ensure fairness, we only compare the results of our experiments on process nodes. A comparison with these detection models is shown in Table 5.

The comparison results show that our model outperforms existing semi-supervised detection models. On the StreamSpot dataset, our model achieves the same experimental results as MAGIC due to the fact that the StreamSpot dataset is scene-separated and does not require much additional information to learn the behavioral patterns of nodes. On the CADETS and TRACE datasets, our model achieves a complete victory, indicating that the additional node features we extract can better describe the behavioral model of nodes during system activities, and showing that the use of the improved GraphMAE2 model can solve the overfitting problem, thus reducing the false positives of the detection results. On Theia data, our model exhibits slightly higher false negatives than MAGIC. In exchange, we further reduce false positives, which is advantageous and can significantly save the working time of security personnel.

5.3. Parameter tuning

In this section, we validate several key parameters in the model, i.e., the dimension d of the node embedding, the number of multi-view re-randomization masks N , and the dimension d_{head} of the multi-head self-attention mechanism. We take the Trace data as an example, and the results of the parameter tuning are shown in Fig. 5.

The tuning experiment for the embedding dimension of the node d is shown in Fig. 5(a). Here we fix $N = 3$, $d_{head} = 96$, and vary d in the range [32-160]. When the embedding dimension is small, the model is unable to adequately capture the complex structures and patterns in the input data. As the embedding dimension increases, the capacity of the model gradually increases to be able to handle more complex feature representations, and the complexity of the model starts to increase, and the model reaches its optimal performance when the embedding dimension is 96. Along with the further increase of the embedding dimension, the optimization difficulty increases, while the introduction of more redundant information leads to a decrease in the detection performance. Therefore, $d = 96$ is selected as the final parameter.

The tuning experiments for the dimension of the multi-head self-attention mechanism d_{head} are shown in Fig. 5(b). Here we fix $N = 3$, $d = 96$, and vary d_{head} in the range [32-160]. The output dimension of the multi-head self-attention mechanism directly determines the richness of the information that the model can capture, and as the output dimension increases, the model is able to capture more information and learn more complex feature representations. When the output dimension is 96, the model obtains the optimal equilibrium point. When the output dimension is too high, the model may overfit the noise and specific details in the training data, resulting in a reduced ability to generalize over unseen data. Therefore, $d_{head} = 96$ is selected as the final parameter.

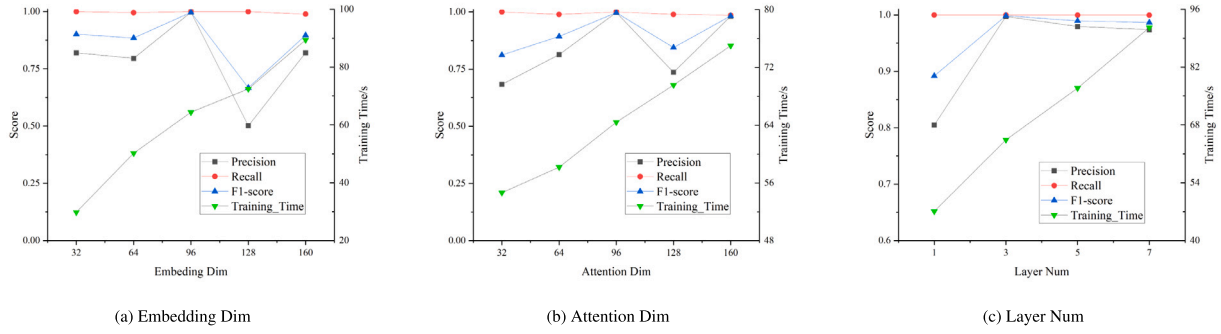


Fig. 5. Parameter tuning.

Table 6
Results of Feature Ablation Experiment.

Features	Precision	Recall	FPR	F1-Score	AUC
D	11.17%	99.77%	29.59%	20.09%	70.41%
F	99.93%	99.73%	0.023%	99.83%	99.98%
T+D	10.08%	99.79%	33.19%	18.31%	66.83%
F+D	62.85%	99.74%	2.19%	97.22%	99.78%
T+F	91.19%	99.81%	0.36%	77.11%	97.79%
T+F+D	98.50%	99.75%	0.05%	99.12%	99.93%

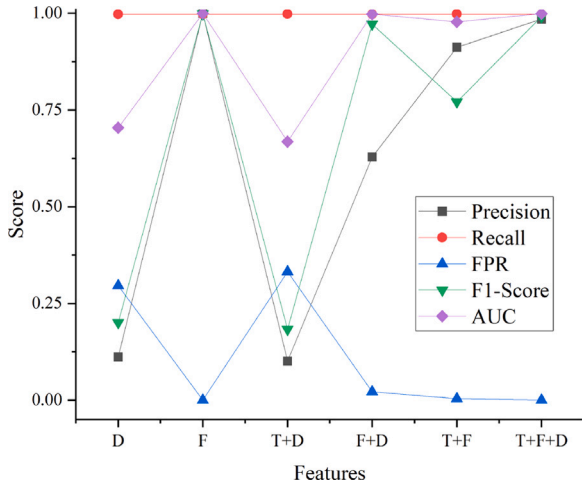


Fig. 6. Feature ablation experiment.

The tuning experiment for multiple views with the number of randomized masks N is shown in Fig. 5(c). Here we fix $d = 96$, $d_{head} = 96$, and vary N in the range [1-7]. When $N = 1$, only one view is randomly masked, which may result in an insufficient regularization effect, and the model may rely too heavily on the information of that single view while ignoring other potentially important information. When the number of random masks is set to 3, this regularization effect may be optimally balanced, neither overemphasizing the information of a single view nor obscuring the information of all views. As N continues to increase, while the regularization effect may be enhanced, too many random masks may introduce too much noise, making it difficult for the model to capture key structures and patterns in the data. Therefore, $N = 3$ was chosen as the final parameter.

5.4. Ablation experiments

In this section, we verify the effect of different modules on the detection results.

Feature ablation: Our model considers three aspects of node features to obtain a comprehensive description of the node behavior

model. Here we evaluate their contributions to entity semantics respectively. Due to space limitations, we take CADETS data as an example. The experimental results are shown in Table 6 & Fig. 6, where D , F , and T represent the type, behavior, and degree of the node, respectively.

The experimental results show that when any of the features are removed, the detection results decrease to varying degrees, suggesting that all the features contain unique patterns of node behavioral cycles. To further illustrate their usefulness, we validate the detection results in different combinations. The experimental results show that all combinations are able to detect the vast majority of malicious nodes (high Recall), which suggests that malicious activity exhibits different interaction characteristics from normal activity. It should be noted that the lowest number of false positives (only 17 false positive points) was achieved in experiments that used only behavioral characteristics of the node, and we did not achieve the same results in other types of data for the same experiments. In order to preserve generality, we still keep all types of feature.

Module ablation: Our model performs semantic fusion with a multi-head self-attention mechanism and uses an improved GraphMAE2 model to learn node features and graph structures to obtain embedded representations of nodes. In this section, we remove or replace each module of the model separately to verify its effectiveness. The ablation comparison model for each module is designed as follows:

Model 1: Stitching together initial embeddings of entities instead of semantic fusion using a multi-head self-attention mechanism;

Model 2: No graph structure reconstruction;

Model 3: Model training using GrapMAE;

Finally, the experimental results of these models are compared with those of the full model.

Fig. 7 shows the experimental results with and without the attention mechanism. The experimental results show that the use of the multi-head self-attention mechanism can reduce the false positives of the detection results. This is because the multi-head self-attention mechanism learns the correlation between the features of different nodes, thus obtaining a canonical expression of the behavioral patterns of the nodes, which is beneficial for the model to learn the embedding of different behaviors.

Fig. 8 illustrates the impact of graph structure reconstruction on the detection results. The experimental results show that in more complex data (TRACE), the introduction of graph structure reconstruction facilitates the model to better learn the difference between different node behaviors. In contrast, in relatively simple data, it enhances the detection of false positives. Relatively speaking, this cost is acceptable and facilitates obtaining more stable detection results.

Fig. 9 shows the impact of GrapMAE2 on the detection results. The experimental results show that GraphMAE2 learns node features from multiple aspects compared to using GraphMAE, which reduces the model's dependence on the discriminability of the input features, and thus improves the robustness and stability of the model, and makes it more resistant to the interference of features.

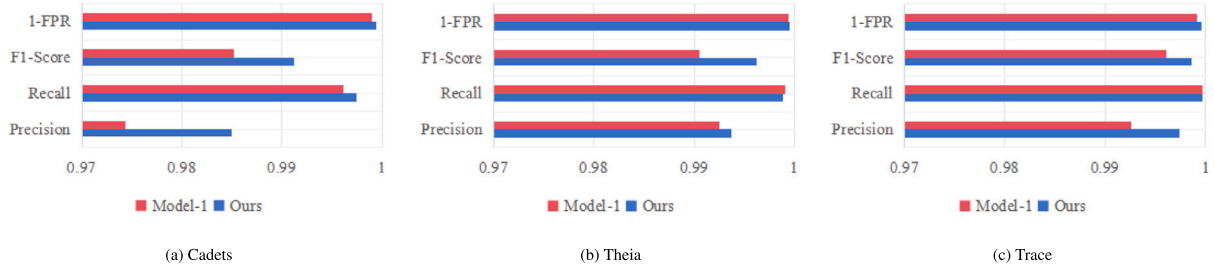


Fig. 7. Impact of self-awareness mechanisms.

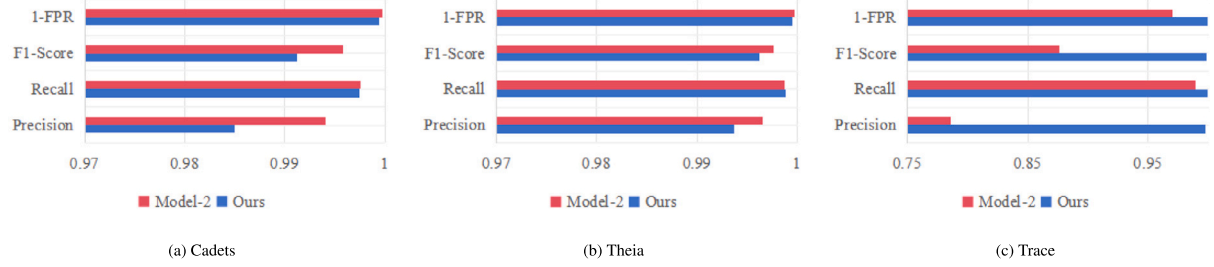


Fig. 8. Impact of structural reconfiguration.

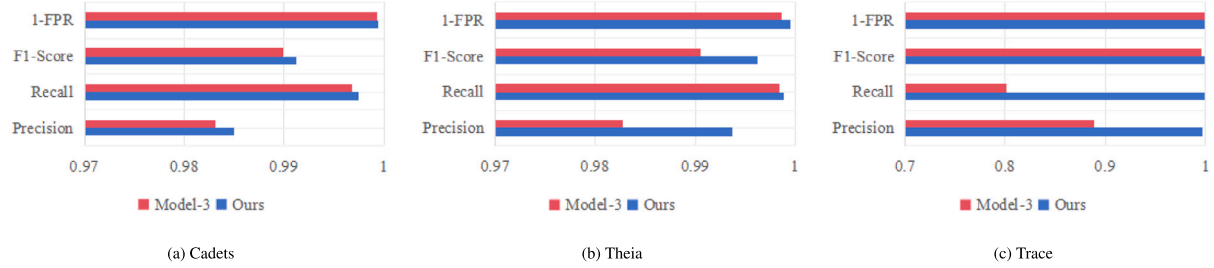


Fig. 9. Impact of GrapMAE2.

5.5. Performance overhead

In this section, we evaluate in detail the time and memory overhead of our proposed model in the training and inference phases. Compared to the baseline model, since we consider richer node features, this leads to a higher embedding dimension and increased memory usage. Given that the number of nodes is N , the number of interactions is E , the node feature dimension is F , the number of interaction types is t , the node embedding dimension is d , the number of layers of the GNN is l , the mask rate is r , and the mask view is n . In the model training phase, the multinomial self-attention mechanism has a time complexity of $O(N^2 \cdot H)$ and a space complexity of $O(d^2 + N \cdot d)$, resulting in a training time of approximately 500 s and a memory consumption of 2300MB, respectively. The masked graph representation learning has a time complexity of $O(k \cdot (E \cdot d^2 + N \cdot d^2) \cdot l + N \cdot d)$, occupying $O((N \cdot F + E \cdot t))$ space to store provenance graphs and $O((N + E) \cdot d)$ space to generate output embeddings.

We tested the performance at runtime of APT-MGL in the E3-Trace subdataset, and the results are shown in Table 7.

The experimental results show that richer features with multi-view embeddings increase the runtime time and memory, but considering that the E3-Trace sub-dataset is about 15 GB of log data collected over a span of several hours, APT-MGL can complete the parsing-to-detection process in about an hour, enabling timely and effective detection.

Table 7

Performance Overhead.

Phase	Time Consumption (s)	Peak Memory Consumption (MB)
Graph Construction	269	6484
Feature Extraction	301	6154
Model Training	2721	11 376
Detection	848	3124

6. Discussion and limitations

Complete data: In this paper, we assume that all current benign system behaviors are observed during the collection of training logs, and model these benign system behaviors to detect APT activity. In our experiments, our model exhibits a low false alarm rate, but has difficulty distinguishing unobserved benign activity. In Section 4.5, we introduce a model adaptation mechanism that adjusts the model parameters through all benign feedbacks to adapt to changing patterns of network behavior. However, this benign feedback-based adaptation mechanism may still have some limitations. For example, the mechanism may be limited in its ability to adapt the model if the benign feedback is not sufficient in relying on feedback data. One possible approach is to introduce an incremental learning mechanism (Li and Hoiem, 2017) that would be used in conjunction with the feedback

mechanism. Incremental learning enables the model to gradually update its own parameters as it receives new data or feedback, without having to rely entirely on a large-scale retraining process.

Attack Recovery: Our model implements node-level detection to achieve accurate localization of attack nodes, but it does not interpret and classify these attack nodes. A viable option is to perform multi-level detection, first performing graph-level detection to determine whether an attack exists, and then further performing node-level detection to determine the specific nodes involved. This has the advantage of reducing the scope of the attack when tracing it back to its origin and facilitates the partitioning of the entities involved in different attacks into corresponding clusters. In addition, recent work (Lv et al., 2024) attempts to classify the different phases of an attack or recover the attack process through machine learning, which by combining these efforts facilitates efficient APT detection and traceability.

Adversarial Attack: In this study, we integrate multiple features such as node types, node actions, and interaction frequencies, and utilize a multi-head self-attention mechanism for feature fusion, which is subsequently processed by an advanced graph representation learning method. This strategy enables the model to capture the complex contextual information and interactions between nodes in the graph more comprehensively, which improves the ability to recognize anomalous behaviors. The advanced graph representation learning method enhances the model's adaptability to changes in complex graph structures and improves its robustness.

We conducted a simple experiment assuming that an attacker changes the local structure of a malicious entity by adding new edges between the malicious process and the benign entity to make its behavior appear more normal. The experimental results show that the model is effective in recognizing malicious activities when dealing with specific types of simulated attacks.

However, recent research (Goyal et al., 2023) has revealed potential challenges of simulation attacks that circumvent detection by inserting benign nodes and edges. For example, simulation attacks exploit strategies such as abusive unweighted graph encoding, distributed graph encoding, and down-sampled graph encoding, which are able to evade detection by reducing the salience of malicious activities without altering the underlying malicious behavior. This exploitation of graph structure manipulation may affect the model's ability to capture global and local graph features.

Although our model has shown a certain degree of robustness in preliminary experiments, the effectiveness of coping with these evasion strategies still needs to be further verified. Future research can consider the following directions:

- Dynamic graph anomaly detection: Combine global and local anomaly detection techniques to dynamically adjust the graph representation to capture subtle structural changes. For example, the dynamic graph anomaly detection method proposed by Karios (Cheng et al., 2023b) effectively identifies subtle anomalies in dynamic graphs by updating the graph representation in real time and combining historical information, thus improving the ability to recognize dynamic attack patterns.
- Weighted graph representation learning: Introduce a weighting mechanism to enhance the recognition of key graph structures and reduce the impact of evasion strategies.
- Adversarial training: EdgeTorrent (King et al., 2023) uses the generative adversarial network (GAN) method and uses adversarial samples for training. The generator generates simulated data, and the discriminator judges the authenticity of the data, thereby improving the robustness of the model to potential attack strategies.

Future work will focus on evaluating the specific impact of these strategies on model performance and exploring effective improvement measures.

7. Conclusion

In this paper, we propose a self-supervised learning-based APT detection model APT-MGL, which learns the embedding representation of nodes based on a new graph mask self-encoder, and transforms the detection problem into an outlier detection problem of malicious nodes to achieve node-level detection. APT-MGL describes the node behavioral patterns from three aspects: node types, node behavioral features, and node degree features, and through the mechanism of multi-head self-attention Perform feature fusion. By learning the behavior patterns of normal data, APT-MGL can detect possible attack activities. The evaluation results show that our model outperforms other node-level APT detection methods and can pinpoint malicious activity-related entities, thus speeding up threat hunting and alert analysis.

CRedit authorship contribution statement

Jiafeng Ren: Writing – review & editing, Writing – original draft, Validation, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Rong Geng:** Writing – review & editing, Resources, Project administration, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

References

- Alsaheel, Abdullah, et al., 2021. {ATLAS}: A sequence-based learning approach for attack investigation. In: 30th USENIX Security Symposium. USENIX Security 21, pp. 3005–3022.
- Alshamrani, Adel, et al., 2019. A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities. IEEE Commun. Surv. Tutor. 21 (2), 1851–1877.
- Anon, 2023a. Darpa transparent computing program engagement3 data release. <https://github.com/darpa-i2o/Transparent-Computing>. (Accessed 20 September 2023).
- Anon, 2023b. The streamspot dataset. <https://github.com/sbustreamspot/sbustreamspot-data>. (Accessed 17 September 2023).
- Bridges, Robert A, et al., 2019. A survey of intrusion detection systems leveraging host data. ACM Comput. Surv. (CSUR) 52 (6), 1–35.
- Chen, Tieming, et al., 2022. Apt-kgl: An intelligent apt detection system based on threat knowledge and heterogeneous provenance graph learning. IEEE Trans. Dependable Secure Comput.
- Cheng, Zijun, et al., 2023a. Ghunter: A fast subgraph matching method for threat hunting. In: 2023 26th International Conference on Computer Supported Cooperative Work in Design. CSCWD, IEEE, pp. 1014–1019.
- Cheng, Zijun, et al., 2023b. Kairos: Practical intrusion detection and investigation using whole-system provenance. arXiv preprint arXiv:2308.05034.
- Devlin, Jacob, et al., 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.
- Die.net, 2020. The linux audit daemon. <https://linux.die.net/man/8/auditd>.
- Gilmer, Justin, et al., 2017. Neural message passing for quantum chemistry. In: International Conference on Machine Learning. PMLR, pp. 1263–1272.
- Goyal, Akul, et al., 2023. Sometimes, you aren't what you do: Mimicry attacks against provenance graph host intrusion detection systems. In: 30th Network and Distributed System Security Symposium.
- Hamilton, Will, Ying, Zhitao, Leskovec, Jure, 2017. Inductive representation learning on large graphs. In: Advances in Neural Information Processing Systems, vol. 30.
- Han, Xueyuan, et al., 2020. Unicorn: Runtime provenance-based detector for advanced persistent threats. arXiv preprint arXiv:2001.01525.
- Hassan, Wajih Ul, Bates, Adam, Marino, Daniel, 2020. Tactical provenance analysis for endpoint detection and response systems. In: 2020 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1172–1189.
- Hassan, Wajih Ul, et al., 2019. Nodoe: Combatting threat alert fatigue with automated provenance triage. In: Network and Distributed Systems Security Symposium.

- Hossain, Md Nahid, Sheikhi, Sanaz, Sekar, R., 2020. Combating dependence explosion in forensic analysis using alternative tag propagation semantics. In: 2020 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1139–1155.
- Hossain, Md Nahid, et al., 2017. {SLEUTH}: Real-time attack scenario reconstruction from {COTS} audit data. In: 26th USENIX Security Symposium. USENIX Security 17, pp. 487–504.
- Hossain, Md Nahid, et al., 2018. {Dependence-Preserving} data compaction for scalable forensic analysis. In: 27th USENIX Security Symposium. USENIX Security 18, pp. 1723–1740.
- Hou, Zhenyu, et al., 2022. Graphmae: Self-supervised masked graph autoencoders. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 594–604.
- Hou, Zhenyu, et al., 2023. Graphmae2: A decoding-enhanced masked self-supervised graph learner. In: Proceedings of the ACM Web Conference 2023. pp. 737–746.
- Jia, Zian, et al., 2023. MAGIC: Detecting advanced persistent threats via masked graph representation learning. arXiv preprint arXiv:2310.09831.
- King, Isaiah J, et al., 2023. EdgeTorrent: Real-time temporal graph representations for intrusion detection. In: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses. pp. 77–91.
- Kipf, Thomas N., Welling, Max, 2016. Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907.
- Li, Linrui, Chen, Wen, 2024. ConGraph: Advanced persistent threat detection method based on provenance graph combined with process context in cyber-physical system environment. Electronics 13 (5), 945.
- Li, Zhizhong, Hoiem, Derek, 2017. Learning without forgetting. IEEE Trans. Pattern Anal. Mach. Intell. 40 (12), 2935–2947.
- Liu, Jian, et al., 2022. A graph learning approach with audit records for advanced attack investigation. In: GLOBECOM 2022-2022 IEEE Global Communications Conference. IEEE, pp. 897–902.
- Luo, Bin, et al., 2024. Detecting APT-exploited processes through semantic fusion and interaction prediction. Comput. Mater. Continua 78 (2).
- Lv, Mingqi, et al., 2024. TREC: APT tactic/technique recognition via few-shot provenance subgraph learning. arXiv preprint arXiv:2402.15147.
- Manzoor, Emaad, Milajerdi, Sadegh M., Akoglu, Leman, 2016. Fast memory-efficient anomaly detection in streaming heterogeneous graphs. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 1035–1044.
- Microsoft Learn, 2024. Event tracing. <https://docs.microsoft.com/en-us/windows/desktop/ETW/event-tracing-portal>.
- Mikolov, Tomas, et al., 2013. Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.
- Milajerdi, Sadegh M, et al., 2019a. Holmes: real-time apt detection through correlation of suspicious information flows. In: 2019 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1137–1152.
- Milajerdi, Sadegh M, et al., 2019b. Poirot: Aligning attack behavior with kernel audit records for cyber threat hunting. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 1795–1812.
- Rehman, Mati Ur, Ahmadi, Hadi, Hassan, Wajih Ul, 2024. FLASH: A comprehensive approach to intrusion detection via provenance graph representation learning. In: 2024 IEEE Symposium on Security and Privacy. SP, IEEE Computer Society, 139–139.
- Shi, Yucheng, et al., 2023. Gigamae: Generalizable graph masked autoencoder via collaborative latent space reconstruction. In: Proceedings of the 32nd ACM International Conference on Information and Knowledge Management. pp. 2259–2269.
- Singla, Ankush, Bertino, Elisa, Verma, Dinesh, 2020. Preparing network intrusion detection deep learning models with minimal data using adversarial domain adaptation. In: Proceedings of the 15th ACM Asia Conference on Computer and Communications Security. pp. 127–140.
- The MITRE Corporation, 2015-2024. MITRE ATT&CK. URL <https://attack.mitre.org>.
- Vaswani, Ashish, et al., 2017. Attention is all you need. In: Advances in Neural Information Processing Systems, vol. 30.
- Veličković, Petar, et al., 2017. Graph attention networks. arXiv preprint arXiv:1710.10903.
- Wang, Qi, et al., 2020. You are what you do: Hunting stealthy malware via data provenance analysis. In: NDSS.
- Wang, Su, et al., 2022. Threatrace: Detecting and tracing host-based threats in node level through provenance graph learning. IEEE Trans. Inf. Forensics Secur. 17, 3972–3987.
- Wikipedia Contributors, 2024. Operation aurora — Wikipedia, the free encyclopedia. Online; URL https://en.wikipedia.org/w/index.php?title=Operation_Aurora&oldid=1223846473. (Accessed 31 May 2024).
- Xiong, Chunlin, et al., 2020. CONAN: A practical real-time APT detection system with high accuracy and efficiency. IEEE Trans. Dependable Secure Comput. 19 (1), 551–565.
- Xu, Zhiqiang, et al., 2022. Depcomm: Graph summarization on system audit logs for attack investigation. In: 2022 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 540–557.
- Yadav, Tarun, Rao, Arvind Mallari, 2015. Technical aspects of cyber kill chain. In: Security in Computing and Communications: Third International Symposium, SSCC 2015, Kochi, India, August 10-13, 2015. Proceedings 3. Springer, pp. 438–452.
- Yan, Na, et al., 2022. DeepPro: Provenance-based APT campaigns detection via GNN. In: 2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications. TrustCom, IEEE, pp. 747–758.
- Yang, Fan, et al., 2023. {PROGRAPHER}: An anomaly detection system based on provenance graph embedding. In: 32nd USENIX Security Symposium. USENIX Security 23, pp. 4355–4372.
- Zengy, Jun, et al., 2022. Shadewatcher: Recommendation-guided cyber threat analysis using system audit records. In: 2022 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 489–506.

Jiafeng Ren received his bachelor's degree from Northeastern University, China in 2022. He is now pursuing his master's degree in Northeastern University, China. His research interests include APT detection and deep learning.

Rong Geng is currently a professor in the School of Computer Science and Engineering at Northeastern University, China. She is mainly engaged in research on wireless networks, space networks, network security and secure routing.